CE 356: Formal Spec and Verification
Term Project: Formal TLAPS Proof

AtLeastOnePrivileged Invariant for Dijkstra's Self-Stabilization Algorithm

Joshua Yao | Spring 2026

1. Why a Formal Proof?

TLC model checking verifies properties for specific parameter values like N=3, K=4. A TLAPS brings it to a whole new level. It is a mathematical argument that the algorithm works for all N and K at once. This is what the project description means by **'prove its correctness.'**

The property proved here is:

> *AtLeastOnePrivileged: in every reachable state, at least one process is privileged.*

This is a safety property, meaning the system can never deadlock. It was chosen because safety properties are significantly easier to prove in TLAPS than liveness properties, but the result is just as important. Without at least one privileged process, the system would be permanently stuck.

2. Proof Strategy

TLAPS proves invariants using the inductive invariant pattern. To prove a property P holds in every reachable state, you need to show:

1. Init => P: P holds in every initial state.
2. P /\ [Next]_x => P': if P holds now and any step occurs, P still holds afterward.

These two conditions, plus temporal reasoning (via the PTL backend), yield Spec => []P. This is the structure used throughout the proof below.

3. The Theorem

The theorem proved is:

```
THEOREM AtLeastOnePrivilegedInvariant ==
  Spec => []AtLeastOnePrivileged
```

Two assumptions are declared at the top of the spec that the proof relies on:

```
ASSUME NAssump == N \in Nat /\ N >= 1
ASSUME KAssump == K \in Nat /\ K >= 2
```

These are the minimum sensible parameters: at least one non-special process and at least two possible counter values.

4. Helper Definitions

Two things are introduced before the main proof. First, TypeOK:

```
TypeOK == x \in [0..N -> 0..K-1]
```

This says x is always a valid state. It is proved alongside AtLeastOnePrivileged throughout and is needed so TLAPS can reason about the type of x.

Second, two LOCAL definitions that mirror Privileged(i) and NumPrivileged but are parameterized over an arbitrary state y instead of x. This is needed to reason about x' (the next state) cleanly in the preservation step.

```
LOCAL PrivState(y, i) == IF i = 0 THEN y[0] = y[N] ELSE y[i] # y[i-1]
LOCAL NumPrivState(y) == Cardinality({i \in 0..N : PrivState(y, i)})
```

5. The ChainArg Lemma

The mathematical core of the proof is:

```
LEMMA ChainArg ==
  \A y \in [0..N -> 0..K-1] : y[0] = y[N] \/ \E i \in 1..N : y[i] # y[i-1]
```

In plain English, this means that for any valid state, either process 0 is privileged (its value equals x[N]), or some adjacent pair in the chain differs (some normal process is privileged). Everything else in the proof comes from this.

5.1 Why It Is True

The intuition is simple. Think about any chain y[0], y[1], ..., y[N]. Either all the values are equal, meaning that y[0] = y[N], or they are not all equal, meaning that somewhere adjacent values must differ. You can't go from one value to a different value without crossing a point of change.

5.2 The Induction

In TLAPS, this is proved by induction on m (the length of the chain so far). Define P(m) as: if m is in 0..N, then y[0] = y[m] or some adjacent pair in 1..m differs. The base case P(0) is y[0] = y[0]. The inductive step splits into two cases:

- If y[0] = y[m], compare y[m] with y[m+1]. Either they are equal (y[0] = y[m+1] by transitivity), or they differ (m+1 is the witness). Either way, P(m+1) holds.
- If some i in 1..m already has y[i] different from y[i-1], then the same i works for P(m+1), since 1..m is a subset of 1..(m+1).

The induction is closed by NatInduction from the TLAPS standard library, then instantiated at m=N. SMT handles the index arithmetic throughout.

6. The StateAtLeastOne Lemma

```
LEMMA StateAtLeastOne ==
  \A y \in [0..N -> 0..K-1] : NumPrivState(y) >= 1
```

This connects ChainArg to the cardinality claim. The proof:

3. Apply ChainArg: either process 0 is privileged or some process i in 1..N is.
4. In either case, exhibit a concrete element of S = {i in 0..N: PrivState(y, i)}, showing S is non-empty.
5. Show S is finite using FS_Interval and FS_Subset from FiniteSetTheorems.
6. Conclude Cardinality(S) >= 1 using FS_EmptySet and SMT arithmetic.

7. The Main Theorem Proof

With both lemmas in hand, the main proof follows the inductive invariant structure:

7.1 Init Establishes the Invariant

```
<1>1. Init => TypeOK /\ AtLeastOnePrivileged
```

Init => TypeOK is immediate since Init picks x from [0..N -> 0..K-1] by definition. TypeOK => AtLeastOnePrivileged follows by applying StateAtLeastOne to x and unfolding the definitions of Privileged and NumPrivileged to match the LOCAL versions.

7.2 Next Preserves the Invariant

```
<1>2. TypeOK /\ AtLeastOnePrivileged /\ [Next]_x => TypeOK' /\ AtLeastOnePrivileged'
```

TypeOK' is proved by case analysis on Next: Act0 produces (x[0]+1) % K which is in range, ActI(i) copies x[i-1] which is already in range, and a stuttering step leaves x unchanged. AtLeastOnePrivileged' then follows by applying StateAtLeastOne to x' exactly as in step <1>1.

7.3 Closing with PTL

```
<1>3. QED
  BY <1>1, <1>2, PTL DEF Spec
```

PTL is the TLAPS temporal logic backend. Given that Init establishes the invariant and Next preserves it, PTL concludes the invariant holds for the full temporal formula Spec = Init /\ [][Next]_x.

8. What This Proof Achieves

The completed proof establishes that for all N >= 1 and K >= 2:

- The system can **never** reach a state with zero privileged processes. So no deadlock ever.
- This holds from any starting state, not just well-formed ones.
- This holds for any ring size N and counter modulus K, not just the specific values TLC checked.

9. Proof Structure Summary

```
ASSUME NAssump, KAssump
 |
 v
LEMMA ChainArg ( induction over chain length)
 |
 v
LEMMA StateAtLeastOne  (cardinality argument using FiniteSetTheorems)
 |
 v
THEOREM AtLeastOnePrivilegedInvariant
 <1>1. Init establishes invariant   (StateAtLeastOne + def unfolding)
 <1>2. Next preserves invariant     (StateAtLeastOne on x')
 <1>3. QED                          (PTL)
```

All proof obligations were verified by TLAPS.

10. Limitations

This proof covers only the safety portion of self-stabilization. The full liveness property is significantly harder to prove in TLAPS and is beyond the scope of this project. The proof sketch document covers the convergence argument in detail. Together, the TLAPS safety proof and the proof sketch give a complete correctness story for the algorithm.